

GUIDA DEFINITIVA — PUNTO 1 (la parte che ti dà la sufficienza)

TdP — Esame pratico Python · MVC + DAO + NetworkX + Flet · v3.0 — sezione 9 con MODEL + CONTROLLER per ogni casistica

Questa guida copre SOLO il Punto 1 (interfaccia DB → costruzione grafo → analisi → stampa), che vale 18–21 e basta per passare. Costruita su tutti i temi fatti insieme: F1 costruttori/circuiti/piloti, BikeStore (vendite + DATEDIFF + ordini + prodotti Fam.1), Chinook, IMDB, FlightDelays, UFO, Genes, Baseball.

La regola che vale più di tutto: un Punto 1 che gira e dà i numeri identici al PDF del prof = sufficienza garantita. Non serve altro.

0. LE 5 DOMANDE — leggi così OGNI traccia

#	Domanda	Cosa decide
1	Il grafo è orientato? (la traccia dice "verso", "uscente/entrante", "dal più X al meno X"?)	<code>nx.DiGraph()</code> se sì, <code>nx.Graph()</code> se no
2	Il nodo ha più di un dato che mi serve dopo? (id + nome + altro?)	Sì → classe <code>@dataclass</code> + <code>idMap</code> . No → stringa diretta
3	C'è un filtro sui nodi? ("età valida", "almeno K vendite", <code>lat/lng...</code>)	Sì → query nodi separata. No → li crea <code>add_edge</code>
4	Come nasce un arco? Il peso/verso dipende dal singolo nodo o dalla relazione tra due?	Famiglia 1 o Famiglia 2 (vedi sez. 1)
5	Quanti dropdown / input e cosa ci metto?	Quante query "riempi menu" mi servono

1. FAMIGLIA 1 vs FAMIGLIA 2 — il cuore di tutto

"Il peso (o il verso) dell'arco dipende da una caratteristica del SINGOLO nodo oppure dalla RELAZIONE tra i due nodi?"

- Dipende dal **singolo nodo** → **FAMIGLIA 1**
- Dipende dalla **relazione tra i due** → **FAMIGLIA 2**

FAMIGLIA 1 — "metrica del singolo nodo"

La traccia dice: "il più venduto", "somma delle vendite dei due", "gare completate da quel team". Query: `SELECT id, SUM(...)` GROUP BY id. In Python: `combinations` + confronto metriche → `add_edge`.

Esempi: Chinook (popolarità artisti), BikeStore vendite/prodotti, F1 circuiti/costruttori.

FAMIGLIA 2 — "relazione diretta"

La traccia dice: "se hanno condiviso un pilota", "se distanti al massimo K giorni", "stessa squadra". Query: `self-join` o tabella ponte → produce già le coppie. In Python: solo `add_edge` in un loop.

Esempi: F1 piloti condivisi, BikeStore ordini (DATEDIFF), Genes, UFO confini.

La regola del SELECT: UN solo id → Famiglia 1. DUE id → Famiglia 2.

```
-- FAMIGLIA 1: un id + la sua metrica
SELECT p.product_id, SUM(oi.quantity) AS vendite GROUP BY p.product_id

-- FAMIGLIA 2: due id + il peso (già coppia)
SELECT c1.constructorId, c2.constructorId, COUNT(*) AS peso
WHERE c1.id < c2.id GROUP BY c1.constructorId, c2.constructorId
```

	FAMIGLIA 1	FAMIGLIA 2
Da dove nasce l'arco	confronto metriche in Python	relazione diretta nel DB
Query SQL	semplice (1 id + metrica)	complessa (self-join, coppie)
Lavoro in Python	<code>combinations</code> + confronto	<code>add_edge</code> in loop
SELECT	1 id + metrica	2 id + peso

Il caso IBRIDO (UFO, Genes)

Arco = Famiglia 2 (relazione diretta), Peso = Famiglia 1 (metrica del singolo nodo). Query 1 nodi con metrica, Query 2 coppie senza peso, Python: `peso = metrica[id1] + metrica[id2]`.

2. QUANDO il calcolo va in SQL e quando in PYTHON

Regola: se riesci a scriverlo come **SELECT COUNT/SUM GROUP BY** testabile su DBeaver → **SQL**. Se ti serve un set, confronto tra nodi, dati già in memoria → **Python**.

- In **SQL**: COUNT(*), SUM(campo), DATEDIFF, (SUM_a+SUM_b)/DATEDIFF
- In **Python**: cromosomi distinti (Genes: set), metrica globale poi confrontata (Fam.1 classica), dati dentro il nodo (nodo.results), pulizie stringa

3. LE QUERY — come capirle e cosa mettere nel SELECT

Tipo	Cosa serve	Cosa restituisce
Dropdown	riempire i menu a tendina	una colonna di valori distinti
Nodi	gli oggetti che diventano nodi del grafo	oggetti (Classe(**row)) o tuple
Archi	relazioni / metriche	coppie + peso (Fam.2) oppure id + metrica (Fam.1)

TRUCCO: scrivi prima il for Python che vorresti avere, poi i ??? che usi sono cosa metti nel SELECT.

```
# Esempio Famiglia 1 — vuoi questo loop:
for prod_id, vendite in righe:
    metrica[prod_id] = vendite
# → SELECT p.product_id AS prod_id, SUM(oi.quantity) AS vendite GROUP BY p.product_id

# Esempio Famiglia 2 — vuoi:
for id1, id2, peso in archi:
    g.add_edge(idMap[id1], idMap[id2], weight=peso)
# → SELECT c1.id AS id1, c2.id AS id2, COUNT(*) AS peso WHERE c1.id < c2.id
```

Query DROPDOWN vuoti

```
SELECT DISTINCT s.shape AS sh FROM sighting s
WHERE s.shape IS NOT NULL AND s.shape <> '' ORDER BY s.shape DESC
-- IS NOT NULL AND <> '' → toglie le voci vuote dal menu (SQL)
-- if val is None: nel controller → controlla che l'utente abbia scelto (Python)
```

Estremi inclusi vs esclusi — leggi la traccia!

```
WHERE ra.year BETWEEN %s AND %s -- inclusi (default F1 A/C)
WHERE ra.year > %s AND ra.year < %s -- esclusi (traccia B)
-- Date in SQL sempre con apici: BETWEEN '2016-01-01' AND '2018-12-28'
```

4. Classe(row) — QUANDO si usa

REGOLA FERREA: i nomi dei campi della dataclass DEVONO combaciare ESATTAMENTE con i nomi/alias delle colonne della SELECT. Se aggiungi SUM(...) AS peso → Classe(**row) crasha! Usa una tupla.

```
# Nodi-oggetto → Classe(**row)
for row in cursor:
    result.append(Prodotto(**row)) # alias SQL = nomi dataclass

# Archi/metriche → TUPLE
result.append((row["id1"], row["id2"], row["peso"]))
result.append((row["prod_id"], row["vendite"]))

# Campo extra "dati dentro il nodo"
results: dict = field(default_factory=dict) # MAI results: dict = {}
```

5. idMap, pesoMap, DUE MAPPE — cosa ci metti dentro

Strumento	Quando
idMap {id: oggetto}	sempre, quando il nodo è una classe e gli archi arrivano come id
pesoMap {id: valore}	Famiglia 1 / ibrido: peso = metrica del nodo
due idMap	due info sullo stesso id da tabelle diverse (Genes)

6. buildGraph — I 6 CASI

Ogni buildGraph inizia SEMPRE con clear() su graph, idMap, e tutte le mappe.

CASO 1 — Grafo completo · Baseball

```
def buildGraph(self, anno):
    self._graph.clear(); self._idMap.clear()
    nodi = DAO.getSquadre(anno)
    self._idMap = {s.id: s for s in nodi}
    self._graph.add_nodes_from(nodi)
    self._graph.add_edges_from(combinations(nodi, 2))
    pesoMap = DAO.getSalari(anno)
    for u, v in self._graph.edges:
        self._graph[u][v]["weight"] = pesoMap[u.id] + pesoMap[v.id]
```

CASO 2 — Famiglia 2 da SQL · F1 piloti, costruttori condivisi

```
def buildGraph(self, y1, y2):
    self._graph.clear(); self._idMap.clear()
    nodi = DAO.getNodi(y1, y2)
    for n in nodi:
        self._idMap[n.id] = n
    self._graph.add_nodes_from(nodi)
    archi = DAO.getArchi(y1, y2) # tuple (id1, id2, peso)
    for id1, id2, peso in archi:
        self._graph.add_edge(self._idMap[id1], self._idMap[id2], weight=peso)
```

CASO 3 — Famiglia 1 combinations · BikeStore prodotti, Chinook

```
def buildGraph(self, categoria, a1, a2):
    self._graph.clear(); self._idMap.clear(); self._vendite.clear()
    nodi = DAO.getNodi(categoria)
    for n in nodi:
        self._idMap[n.product_id] = n
    self._graph.add_nodes_from(nodi)
    righe = DAO.getVendite(categoria, a1, a2) # tuple (prod_id, SUM(quantity))
    for prod_id, tot in righe:
        self._vendite[prod_id] = tot
    # combinations SOLO sui nodi attivi (quelli con una metrica)
    for a, b in combinations(self._vendite.keys(), 2):
        peso = self._vendite[a] + self._vendite[b]
        if self._vendite[a] > self._vendite[b]:
            self._graph.add_edge(self._idMap[a], self._idMap[b], weight=peso)
        elif self._vendite[a] < self._vendite[b]:
            self._graph.add_edge(self._idMap[b], self._idMap[a], weight=peso)
        else: # pareggio → DUE archi (solo se DiGraph)
            self._graph.add_edge(self._idMap[a], self._idMap[b], weight=peso)
            self._graph.add_edge(self._idMap[b], self._idMap[a], weight=peso)
```

CASO 4 — Dati DENTRO il nodo · F1 circuiti/costruttori

```
def buildGraph(self, y1, y2):
    self._graph.clear(); self._idMap.clear()
    nodi = DAO.getTuttiCostruttori() # TUTTI, senza filtro anni
    for n in nodi: self._idMap[n.constructorId] = n
    self._graph.add_nodes_from(nodi)
    for cId, anno, dId, pos in DAO.getPiazzamenti(y1, y2):
        if anno not in self._idMap[cId].results:
            self._idMap[cId].results[anno] = []
        self._idMap[cId].results[anno].append((dId, pos))
    attivi = [n for n in self._graph.nodes() if len(n.results) > 0]
    for a, b in combinations(attivi, 2):
        self._graph.add_edge(a, b, weight=self.gareCompletate(a)+self.gareCompletate(b))

def gareCompletate(self, nodo):
    return sum(len(lista) for lista in nodo.results.values())
```

Trappola GROUP BY: non aggiungere position al GROUP BY — ogni riga è già unica per (racelId, driverId).

CASO 5 — Due mappe (idMap + pesoMap) · UFO

```
def buildGraph(self, lat, lon, shape):
    self._graph.clear(); self._idMap.clear(); self._pesoMap.clear()
    for s in DAO.getTuttiStati(): self._idMap[s.id] = s
    for id, peso in DAO.getNodiAttivi(lat, lon, shape): self._pesoMap[id] = peso
    for id in self._pesoMap: self._graph.add_node(self._idMap[id])
    for id1, id2 in DAO.getConfini():
        if id1 in self._pesoMap and id2 in self._pesoMap:
            self._graph.add_edge(self._idMap[id1], self._idMap[id2],
                                weight=self._pesoMap[id1]+self._pesoMap[id2])
```

◆ NUOVO — CASO 6 — DiGraph orientato per DATA/TEMPO + peso DATEDIFF · BikeStore ordini 2025-11-10

Stesso schema del Caso 2 ma DiGraph. Il verso è garantito dal SQL (`o1.date < o2.date`). Il peso è una formula con divisione.

```
# SQL — la query degli archi fa il lavoro pesante:
WHERE o1.order_date < o2.order_date          -- verso: vecchio → recente
  AND DATEDIFF(o2.order_date, o1.order_date) <= %s -- max K giorni
# peso = (SUM(o1.quantity) + SUM(o2.quantity)) / DATEDIFF(o2.date, o1.date)

# buildGraph — identico al Caso 2, grafo è DiGraph:
def buildGraph(self, store, k):
    self._graph.clear(); self._idMap.clear()
    for n in DAO.getNodi(store): self._idMap[n.order_id] = n
    self._graph.add_nodes_from(list(self._idMap.values()))
    for id1, id2, peso in DAO.getArchi(store, k):
        self._graph.add_edge(self._idMap[id1], self._idMap[id2], weight=peso)
```

DATEDIFF nel denominatore non può essere 0 se hai `o1.date < o2.date` (stretto). Se usi `<=`, aggiungi `AND DATEDIFF(...) > 0` nel WHERE.

Come scegli il caso

- "tutti con tutti" → Caso 1
- arco da self-join / tabella ponte con peso già in SQL → Caso 2
- "il più X" + metrica del singolo nodo → Caso 3
- "salva dentro il nodo i risultati" → Caso 4
- filtro nodi + tabella ponte per gli archi + peso = metrica nodo → Caso 5
- arco da data vecchia → data recente, peso = formula / DATEDIFF → Caso 6

7. LA CLASSE NODO — sempre @dataclass con 3 metodi

```
from dataclasses import dataclass, field

@dataclass
class Constructor:
    constructorId: int    # nomi = alias SQL esatti
    name: str
    results: dict = field(default_factory=dict) # solo se "dati dentro il nodo"

    def __hash__(self): return hash(self.constructorId) # solo chiave primaria
    def __eq__(self, other): return self.constructorId == other.constructorId
    def __str__(self): return f"{self.name}" # solo ciò che vuoi stampare
```

8. IL CONTROLLER — validazione + COME STAMPARE i valori

L'ordine SACRO della validazione

```
def handleCreaGrafo(self, e):
    val = self._view._dd.value
    if val is None:          # 1) VUOTO?
        self._view.txt_result.controls.clear()
        self._view.txt_result.controls.append(ft.Text("Selezionare un valore", color="red"))
        self._view.update_page(); return
    try:                     # 2) CONVERSIONE
        x = int(val)
    except ValueError:
        self._view.txt_result.controls.clear()
        self._view.txt_result.controls.append(ft.Text("Inserire un numero", color="red"))
        self._view.update_page(); return
    if not (minimo <= x <= massimo): # 3) RANGE
        self._view.txt_result.controls.clear()
        self._view.txt_result.controls.append(ft.Text("Valore fuori range", color="red"))
        self._view.update_page(); return
    self._model.buildGraph(val) # 4) MODEL + stampa
    self._view.txt_result.controls.clear()
    self._view.update_page()
```

REGOLA FERREA: ogni return di errore deve avere update_page() PRIMA. E clear() prima dei risultati finali, altrimenti si accumulano.

COME STAMPARE — le 3 forme base

A) Arco da edges(data=True) → tupla (nodo1, nodo2, {"weight": ...})

```
for a in top5:
    self._view.txt_result.controls.append(
        ft.Text(f"{a[0]} -> {a[1]} ({a[2]['weight']})")
    #      ^nodo1   ^nodo2   ^peso (virgolette SINGOLE!)
```

B) Nodo-oggetto → attributi con il punto

```
for n in nodi:
    self._view.txt_result.controls.append(ft.Text(f"{n.name} -- {n.score}"))
```

C) Tupla (nodo, valore) → unpacking nel for

```
for nodo, peso in lista:
    self._view.txt_result.controls.append(ft.Text(f"{nodo.name} -- {peso}"))
# peso è già un numero, NON peso['weight']!
```

Popolare un secondo dropdown dopo aver creato il grafo

```
# Alla fine di handleCreaGrafo:
self._view._ddNode.options.clear() # .options.clear(), NON .clear()
for n in self._model.getAllNodes():
    self._view._ddNode.options.append(ft.dropdown.Option(n.order_id))
self._view._ddNode.disabled = False
self._view._btnCerca.disabled = False
self._view.update_page()
# MAI nella load_interface (il grafo non esiste ancora)
```

♦ NUOVO — D) Multi-attributo del nodo — accedi a PIÙ campi esplicitamente

Quando il formato richiede più campi (es. driverRef (driverId) -- DoB: dob (grado=N)):

```
for n, grado in lista:
    self._view.txt_result.controls.append(
        ft.Text(f"{n.driverRef} ({n.driverId}) -- DoB: {n.dob} (grado={grado})"))
# Regola: __str__ → solo se vuoi SOLO il campo di __str__.
# Se vuoi più info → accedi agli attributi esplicitamente.
```

♦ NUOVO — E) Intestazione componente più grande con conteggio

```
maggiore = self._model.getMaggioreOrdinata() # lista di (nodo, grado)
self._view.txt_result.controls.append(
    ft.Text(f"Componente piu grande ({len(maggiore)} nodi):"))
for n, m in maggiore:
    self._view.txt_result.controls.append(
        ft.Text(f"{n.driverRef} ({n.driverId}) -- DoB: {n.dob} (grado={m})"))
# len(maggiore) = dimensione componente, senza query extra
```

9. PUNTO C / D — CASISTICHE DI STAMPA (MODEL + CONTROLLER)

Per ogni casistica: prima il codice del **model** (sfondo grigio), poi come chiamarlo e stamparlo nel **controller** (sfondo verde). **Items 14-17 = nuovi.**

1) Numero di nodi e archi

MODEL

```
def graphDetails(self):
    return len(self._graph.nodes), len(self._graph.edges)
```

CONTROLLER

```
nodi, archi = self._model.graphDetails()
self._view.txt_result.controls.append(ft.Text(f"Numero di nodi:{nodi}"))
self._view.txt_result.controls.append(ft.Text(f"Numero di archi:{archi}"))
```

2) Top 5 archi per peso (decrescente)

MODEL

```
def getTop5Archi(self):
    return sorted(self._graph.edges(data=True),
                  key=lambda x: x[2]["weight"], reverse=True)[:5]
# Per crescente → reverse=False. Alcuni temi (Genes) chiedono crescente!
```

CONTROLLER

```
top5 = self._model.getTop5Archi()
self._view.txt_result.controls.append(ft.Text("Archi di peso maggiore:"))
for a in top5:
    self._view.txt_result.controls.append(
        ft.Text(f"{a[0]} -> {a[1]} ({a[2]['weight']}"))
```

3) Top 5 nodi per GRADO (numero di archi)

MODEL

```
def getTop5Nodi(self):
    return sorted([(n, self._graph.degree(n)) for n in self._graph.nodes()],
                  key=lambda x: x[1], reverse=True)[:5]
```

CONTROLLER

```
top5 = self._model.getTop5Nodi()
for nodo, grado in top5:
    self._view.txt_result.controls.append(
        ft.Text(f"{nodo} -> degree: {grado}"))
```

4) Top 5 nodi per ARCHI USCENTI (DiGraph)

MODEL

```
def getTop5Uscenti(self):
    return sorted([(n, self._graph.out_degree(n)) for n in self._graph.nodes()],
                  key=lambda x: x[1], reverse=True)[:5]
# in_degree(n) per gli entranti
```

CONTROLLER

```
top5 = self._model.getTop5Uscenti()
for nodo, grado in top5:
    self._view.txt_result.controls.append(
        ft.Text(f"{nodo} -> out_degree: {grado}"))
```

5) Componenti connesse (Graph non orientato)

MODEL

```
def getComponenti(self):
    comps = list(nx.connected_components(self._graph))
    return sorted([c for c in comps if len(c) > 1], key=len, reverse=True)
```

CONTROLLER

```
comps = self._model.getComponenti()
self._view.txt_result.controls.append(ft.Text(f"{len(comps)} componenti con >1 nodo"))
for c in comps:
    nomi = ", ".join([n.name for n in c])
    self._view.txt_result.controls.append(ft.Text(f"{nomi} | dim={len(c)}"))
```

6) Componenti DEBOLMENTE connesse (DiGraph)

Su un DiGraph `connected_components` NON esiste! Usa quelle deboli.

MODEL

```
def getComponentiDeboli(self):
    comps = list(nx.weakly_connected_components(self._graph))
    return sorted(comps, key=len, reverse=True)

def getNumeroComponenti(self):
    return nx.number_weakly_connected_components(self._graph)
```

CONTROLLER

```
n = self._model.getNumeroComponenti()
self._view.txt_result.controls.append(ft.Text(f"Il grafo ha {n} componenti connesse"))
```

7) Componente connessa PIÙ GRANDE

MODEL

```
def getMaggiore(self):
    return max(nx.connected_components(self._graph), key=len) # set di nodi
```

CONTROLLER

```
maggiore = self._model.getMaggiore()
self._view.txt_result.controls.append(
    ft.Text(f"Componente più grande: {len(maggiore)} nodi"))
```

8) Componente che contiene UN nodo specifico

MODEL

```
def getCompDi(self, source):
    return nx.node_connected_component(self._graph, source) # set
```

CONTROLLER

```
nodo_val = self._view._ddNode.value
source = self._model._idMap[int(nodo_val)]
comp = self._model.getCompDi(source)
self._view.txt_result.controls.append(ft.Text(f"Componente di {source}: {len(comp)} nodi"))
```

9) Nodi della componente più grande, ordinati per peso MAX arco incidente

La richiesta "difficile" dei temi F1 costruttori. Il numero accanto al nome è il peso massimo tra gli archi che toccano quel nodo.

MODEL

```
def getDettagliComponente(self):
    largest = max(nx.connected_components(self._graph), key=len)
    ordinati = sorted(largest,
                      key=lambda v: max(d["weight"]
                                         for _, _, d in self._graph.edges(v, data=True)),
                      reverse=True)
    return [(v, max(d["weight"] for _, _, d in self._graph.edges(v, data=True)))
            for v in ordinati]
# Per il peso MINIMO: sostituisci max con min
```

CONTROLLER

```
lista = self._model.getDettagliComponente()
for nodo, peso_max in lista:
    self._view.txt_result.controls.append(
        ft.Text(f"{nodo} -- peso_max: {peso_max}"))
```

10) Vicini di un nodo, ordinati per peso

MODEL

```
def getVicini(self, source):
    out = [(v, self._graph[source][v]["weight"]) for v in self._graph.neighbors(source)]
    out.sort(key=lambda x: x[1], reverse=True)
    return out
```

CONTROLLER

```
source = self._model._idMap[int(self._view._ddNode.value)]
vicini = self._model.getVicini(source)
for nodo, peso in vicini:
    self._view.txt_result.controls.append(ft.Text(f"{nodo} -- peso: {peso}"))
```

11) Nodo con più influenza (DiGraph): peso uscenti – peso entranti

MODEL

```
def getBestNode(self):
    best, bestScore = None, None
    for v in self._graph.nodes():
        out = sum(d["weight"] for _, _, d in self._graph.out_edges(v, data=True))
        inn = sum(d["weight"] for _, _, d in self._graph.in_edges(v, data=True))
        score = out - inn
        if bestScore is None or score > bestScore:
            bestScore, best = score, v
    return best, bestScore
```

CONTROLLER

```
best, score = self._model.getBestNode()
self._view.txt_result.controls.append(
    ft.Text(f"Nodo più influente: {best} (score={score})"))
```

12) Cammino massimo (DFS)

MODEL

```
import copy
def getCammino(self, source_id):
    source = self._idMap[int(source_id)]
    lp = []
    tree = nx.dfs_tree(self._graph, source)
    for node in tree.nodes():
        tmp = [node]
        while tmp[0] != source:
            pred = nx.predecessor(tree, source, tmp[0])
            tmp.insert(0, pred[0])
        if len(tmp) > len(lp):
            lp = copy.deepcopy(tmp)
    return lp
```

CONTROLLER

```
nodo_val = self._view._ddNode.value
if nodo_val is None:
    self._view.txt_result.controls.append(ft.Text("Selezionare un nodo", color="red"))
    self._view.update_page(); return
cammino = self._model.getCammino(nodo_val)
self._view.txt_result.controls.append(ft.Text(f"Nodo di partenza: {nodo_val}"))
for n in cammino:
    self._view.txt_result.controls.append(ft.Text(f"{n}"))
```

13) join per stampare un set di nodi su una riga

MODEL

```
# Nel model non serve un metodo separato – si fa nel controller direttamente
# Usa: ", ".join([n.GeneID for n in componente])
```

CONTROLLER

```
comp = self._model.getMaggiore() # set di nodi
nomi = ", ".join([n.GeneID for n in comp])
self._view.txt_result.controls.append(ft.Text(f"{nomi} | dim={len(comp)}"))
```

◆ NUOVO — 14) Nodi della componente più grande ordinati per GRADO · [F1 piloti 2025-09-15]

Differenza dal caso 9: caso 9 ordina per max peso arco. Questo ordina per $\text{degree}(n)$ = numero di archi del nodo. Leggi la traccia: 'grado' → caso 14; 'peso massimo arco' → caso 9.

MODEL

```
def getMaggioreOrdinata(self):
    largest = max(nx.connected_components(self._graph), key=len)
    ordinati = sorted(largest,
                      key=lambda n: self._graph.degree(n),
                      reverse=True)
    return [(n, self._graph.degree(n)) for n in ordinati]
```

CONTROLLER

```
maggiore = self._model.getMaggioreOrdinata()
self._view.txt_result.controls.append(
    ft.Text(f"Componente piu grande ({len(maggiore)} nodi):"))
for n, m in maggiore:
    # formato semplice:
    self._view.txt_result.controls.append(ft.Text(f"{n} (grado={m})"))
    # oppure multi-attributo (es. F1 piloti):
    # self._view.txt_result.controls.append(
    #     ft.Text(f"{n.driverRef} ({n.driverId}) -- DoB: {n.dob} (grado={m})"))
```

◆ NUOVO — 15) Top 5 nodi per SCORE (inn - out) · [BikeStore Traccia B prodotti]

La traccia dice: 'i 5 prodotti più venduti = nodi la cui somma archi entranti meno archi uscenti è massima'. Diverso dal caso 11 che fa out-inn (influenza). Qui è inn-out (popolarità).

MODEL

```
def getTop5Score(self):
    scores = []
    for v in self._graph.nodes():
        inn = sum(d["weight"] for _, _, d in self._graph.in_edges(v, data=True))
        out = sum(d["weight"] for _, _, d in self._graph.out_edges(v, data=True))
        score = inn - out # ← inn - out (non out - inn!)
        scores.append((v, score))
    return sorted(scores, key=lambda x: x[1], reverse=True)[:5]
```

CONTROLLER

```
top5 = self._model.getTop5Score()
self._view.txt_result.controls.append(ft.Text("I cinque prodotti piu venduti sono:"))
for nodo, score in top5:
    self._view.txt_result.controls.append(
        ft.Text(f"{nodo.product_name} - {nodo.model_year} with score {score}"))
```

◆ NUOVO — 16) getAllNodes — per popolare il secondo dropdown

Pattern completo da usare alla fine di handleCreaGrafo per abilitare il dropdown dei nodi.

MODEL

```
def getAllNodes(self):
    return self._graph.nodes() # restituisce i nodi oggetto del grafo
```

CONTROLLER

```
# Alla fine di handleCreaGrafo, dopo buildGraph:
self._view._ddNode.options.clear()
for n in self._model.getAllNodes():
    self._view._ddNode.options.append(ft.dropdown.Option(n.order_id))
    # oppure n.product_id, n.driverId, ecc. — la chiave primaria del nodo
self._view._ddNode.disabled = False
self._view._btnCerca.disabled = False
self._view.update_page()
```

◆ NUOVO — 17) Verso DATA/TEMPO + numero componenti (DiGraph) · [BikeStore ordini]

Quando il grafo è DiGraph orientato per data, le componenti si chiamano con `weakly_connected_components`.

MODEL

```
def getNumComponenti(self):
    return nx.number_weakly_connected_components(self._graph)
```

CONTROLLER

```
n = self._model.getNumComponenti()
self._view.txt_result.controls.append(
    ft.Text(f"Il grafo ha {n} componenti connesse (deboli)"))
```

10. NETWORKX — cheat-sheet

```
import networkx as nx
g = nx.Graph() # non orientato
g = nx.DiGraph() # orientato

g.add_node(x); g.add_nodes_from(lista) # lista di oggetti, NON di chiavi!
g.add_edge(u, v, weight=w) # crea anche i nodi se mancano
g.add_edges_from(combinations(L, 2)) # tutti con tutti
g[u][v]["weight"] # peso arco
g.clear(); len(g.nodes); len(g.edges)

# ITERARE
g.edges(data=True) # (u, v, {"weight": w})
g.neighbors(x) # vicini (Graph)
g.degree(x) # grado totale
g.out_edges(x, data=True) # uscenti (DiGraph)
g.in_edges(x, data=True) # entranti (DiGraph)
g.out_degree(x) / g.in_degree(x)

# COMPONENTI — Graph
nx.number_connected_components(g)
nx.connected_components(g) # iterabile di set
max(nx.connected_components(g), key=len) # la più grande
nx.node_connected_component(g, source)

# COMPONENTI — DiGraph → DEBOLI!
nx.number_weakly_connected_components(g)
nx.weakly_connected_components(g)

# PERCORSI
nx.dfs_tree(g, source)

# ORDINAMENTI
sorted(g.edges(data=True), key=lambda x: x[2]["weight"], reverse=True)[:5]
sorted(g.nodes(), key=lambda n: g.degree(n), reverse=True)
```

11. SQL — i mattoncini

Mattoncino	Quando	Esempio
SELECT DISTINCT	evitare duplicati	SELECT DISTINCT s.shape
GROUP BY + COUNT(*)	contare per gruppo	GROUP BY x.id
HAVING	filtro DOPO aggregato	HAVING N >= %s
BETWEEN %s AND %s	range inclusi	year BETWEEN %s AND %s
> %s AND < %s	range esclusi	year > %s AND year < %s
self-join id1 < id2	coppie distinte (Fam.2)	WHERE t1.id < t2.id
DATEDIFF(d2, d1)	giorni tra due date	peso o condizione
SUM(q1+q2)/DATEDIFF	peso formula/divisione	BikeStore ordini
SUM(campo)	somma pezzi	SUM(oi.quantity)
IS NOT NULL / <> "	togliere vuoti dal menu	position IS NOT NULL
backtick `year`	nomi colonna riservati	t.`year` = %s
%s + tupla	parametri sicuri	cursor.execute(q, (x,))

WHERE vs HAVING: filtro su aggregato → HAVING. Campo normale → WHERE.

12. ERRORI CHE FAI SEMPRE — checklist pre-consegna

```
[ ] set scritto al posto di self → self._view...
[ ] add_nodes_from invece di add_edge per gli ARCHI
[ ] add_nodes_from(idMap) aggiunge le chiavi → usa idMap.values()
[ ] return di errore senza update_page() prima → l'errore non si vede
[ ] manca clear() prima di stampare → i risultati si accumulano
[ ] .clear() invece di .options.clear() sui dropdown
[ ] virgolette doppie dentro f-string → usa singole: f"{e[2]['weight']}"
[ ] validazione nell'ordine sbagliato → vuoto → converti → range → model
[ ] int() / float() senza try/except
[ ] parametro e mancante negli handler → def handleXxx(self, e):
[ ] parentesi mancanti sulla chiamata → self._model.getX() non self._model.getX
[ ] buildGraph() senza il parametro del filtro
[ ] results: dict = {} invece di field(default_factory=dict)
[ ] accedere a [id1] con le tonde (id1) invece delle quadre [id1]
[ ] componenti su DiGraph con connected_components → usa weakly_connected_components
[ ] rimuovere nodi isolati quando la traccia dice 'rimangono isolati' → NON rimuoverli
[ ] GROUP BY con campo di troppo (es. position) che falsa il conteggio
[ ] alias SQL ≠ nomi dataclass → Classe(**row) crasha
[ ] max / min / list usati come nome di variabile → sovrascrivono i built-in
[ ] DATEDIFF nel denominatore senza controllo > 0 → ZeroDivisionError
[ ] estremi inclusi vs esclusi → leggi la traccia (BETWEEN vs > AND <)
[ ] ordinamento per GRADO confuso con ordinamento per PESO ARCO (leggi la traccia!)
[ ] score inn-out confuso con out-inn → inn-out = popolarità, out-inn = influenza
[ ] multi-attributo nodo: usare solo __str__ quando serve driverRef + driverId + dob
[ ] _idRiga / _mapVendite.clear() mancante → i dati si accumulano tra esecuzioni
```

13. PROCEDURA CRONOMETRATA (2 ore)

Tempo	Cosa fai
0–10 min	Leggi la traccia. 5 domande. Identifica Famiglia e Caso buildGraph (1–6).
10–20 min	Query su DBeaver con valori fissi. Verifica: dropdown + nodi + archi.
20–30 min	DAO: incolla query testate, metti %s.
30–35 min	Classe Nodo (@dataclass + hash/eq/str). idMap nel model.
35–55 min	buildGraph. Testa con testModel (nodi/archi) prima della GUI.
55–65 min	Metodi sez. 9: graphDetails, top-K, componenti.
65–80 min	View + Controller (fillDD + handleCreaGrafo con validazione).
80–90 min	LANCIA. Confronta numeri con PDF prof. Debugga.
90–105 min	Checklist sez. 12. Sistema i typo.
105–120 min	Consegna.

Priorità: Punto 1 che gira con numeri giusti = 18–21. Sempre testa su DBeaver e con testModel prima della GUI.

RIEPILOGO IN UNA PAGINA

- **5 domande** → orientato? nodo classe? filtro nodi? Famiglia 1 o 2? quanti input?
- **Famiglia 1** = metrica del singolo nodo → query semplice (1 id) + combinations. **Famiglia 2** = relazione diretta → query con coppie (2 id) + add_edge loop.
- **SELECT**: scrivi prima il for Python, i ??? che usi sono cosa metti nel SELECT.
- **Dropdown vuoti**: IS NOT NULL AND <> " nel menu; if val is None nel controller.
- **Classe(**row)** per nodi-oggetto (alias = campi). **Tuple** per archi/metriche.
- **idMap**={id:oggetto}, **pesoMap**={id:valore}, due mappe se due tabelle.
- **6 casi buildGraph**: completo / Fam.2 SQL / Fam.1 combinations / dati nel nodo / due mappe / DiGraph DATEDIFF.
- **Sez.9 — stampa arco**: a[0], a[1], a[2]['weight'] · **oggetto**: n.attr · **tupla**: for nodo,val in lista.
- **Score**: inn-out = popolarità (BikeStore prodotti) · out-inn = influenza (getBestNode).
- **Componenti**: Graph → connected_components · DiGraph → weakly_connected_components.
- **Grado vs peso arco**: degree(n) → grado · max(d['weight'] for ...) → peso max arco.
- **Valida sempre**: vuoto → converti → range → model. update_page() prima di ogni return.

In bocca al lupo Armi. Hai fatto il lavoro: ora è solo ripetere finché gli errori della sez. 12 spariscono.